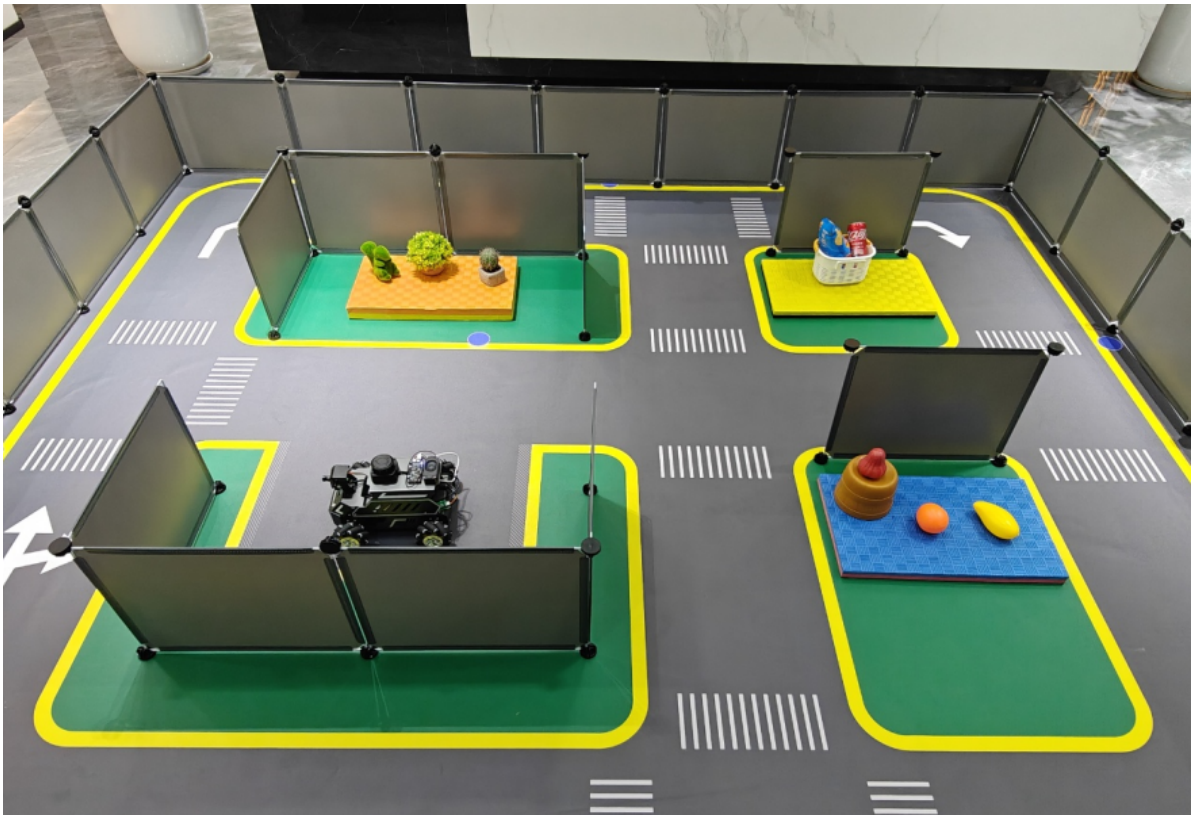
【Track Map】Multimodal visual understand+SLAM navigation

1. Course Content

1. Learn to use the robot's visual understanding combined with SLAM navigation on a Track map.

2. Preparation



2.1 Content Description

> This lesson uses the Orin, USB camera PTZ version user as an example. For Raspberry Pi and Jetson-Nano boards, you need to open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this lesson in the terminal. For instructions on entering the Docker

> container from the host computer, refer to **[01. Robot Configuration and Operation Guide] -- [4.Enter Docker (For JETSON Nano and RPi 5)]**. For Orin boards, simply open a terminal and enter the commands mentioned in this lesson.

This example uses `model:"qwen/qwen2.5-vl-72b-instruct:free","qwen-vl-latest"`

⚠ The responses from the big model for the same test command may not be exactly the same and may differ slightly from the screenshots in the tutorial. To increase or decrease the diversity of the big model's responses, refer to the section on configuring the decision-making big model parameters in **[03.AI Model Basics] -- [5.Configure AI large model]** .

2.2 Configuring the Mapping File

For detailed information on the principles and concepts of map mapping, please refer to the course content in [2. AI Large Model Basics - 3. Embodied Intelligent Robot System Architecture].

Before running this example, you need to construct and save a map using one of the following algorithms from the **[LiDAR Course]-[5.Gmapping-SLAM mapping], [6.Cartographer-SLAM mapping], [7.slam_toolbox mapping]** , resulting in a .yaml map file.

For Raspberry Pi and Jetson-Nano boards, you need to enter the Docker container first. For Orin controllers, this is not necessary.

Enter the Docker container (see [Docker course] --- [4. Docker Startup Script] for steps).

All the following commands must be executed within the same Docker container **(see [Docker course] --- [3. Docker Submission and Multi-Terminal Access] for steps).**

Connect to the robot's desktop via VNC, open the terminal, and enter the command.

```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```

Start the virtual machine by creating a new terminal. **(For Raspberry Pi, and Jetson Nano, it is recommended to run the visualization on the virtual machine.)**

```
ros2 launch yahboomcar_nav display_nav_launch.py
```

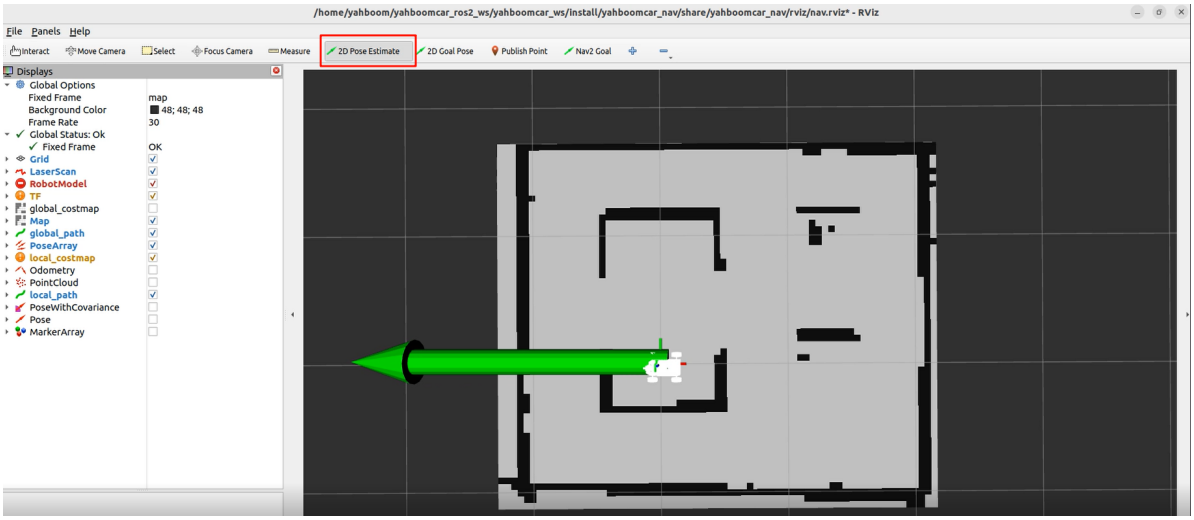
Wait for the navigation algorithm to activate and the map to appear, then open the terminal and enter.

```
# Choose one of the two navigation algorithms
# Standard navigation
ros2 launch yahboomcar_nav navigation_teb_launch.py

#Fast relocalization navigation ( jetson nano and Raspberry Pi 5 are not supported)
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false load_state_filename:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.pbstream

ros2 launch yahboomcar_nav navigation_cartodwb_launch.py maps:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.yaml params_file:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/params/cartoteb_nav_params.yaml
```

Note: yahboomcar.yaml and yahboomcar.pbstream must be mapped simultaneously, meaning they are the same map. Refer to the cartograph mapping algorithm for saving maps.

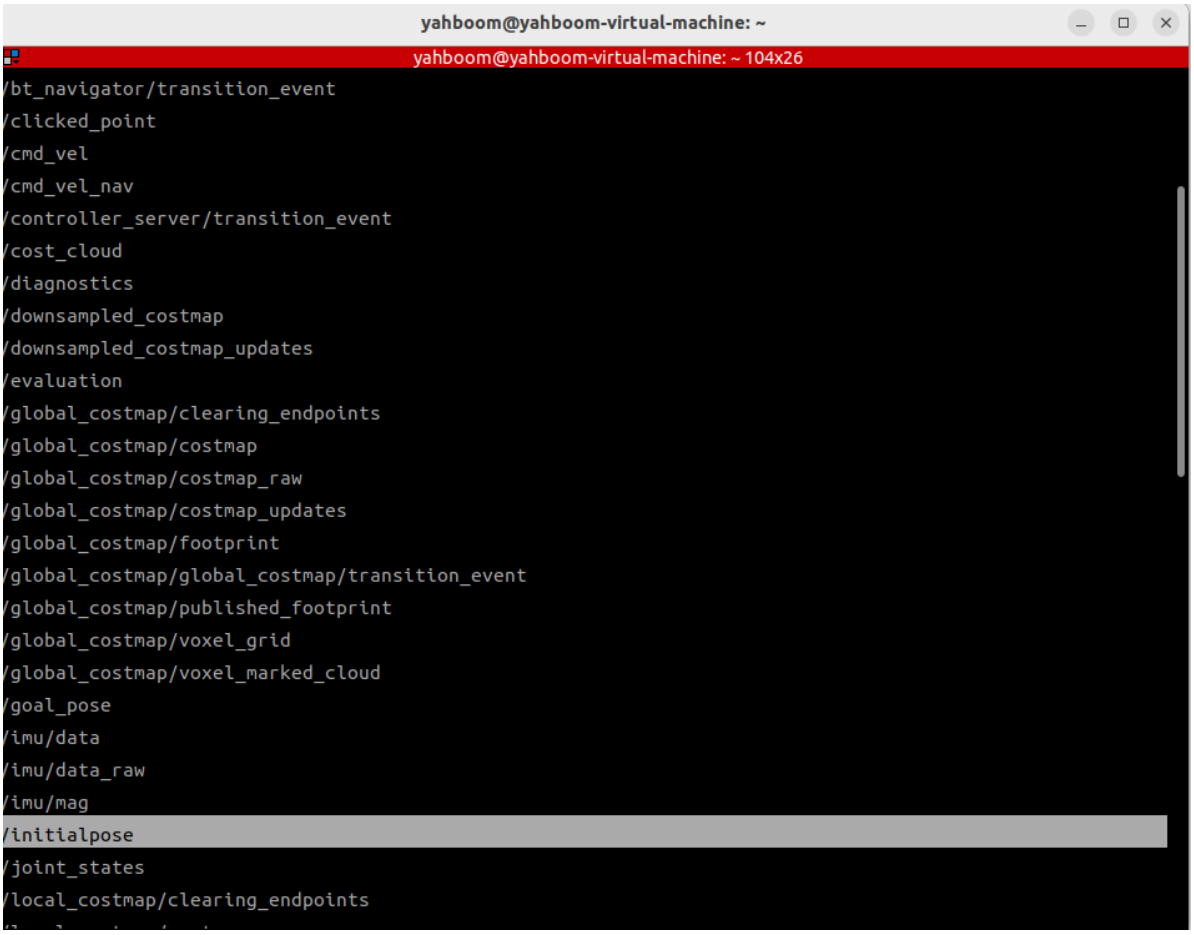
After that, follow the navigation function startup process to initialize positioning. The rviz2 visualization interface will open. Click **2D Pose Estimate** in the upper toolbar to enter the selection state. Roughly mark the robot's position and orientation on the map. After initializing positioning, preparations are complete.

The robot's model will be displayed on the map, as shown in the image below:



Create another terminal in the virtual machine and enter the command.

```
ros2 topic list
```

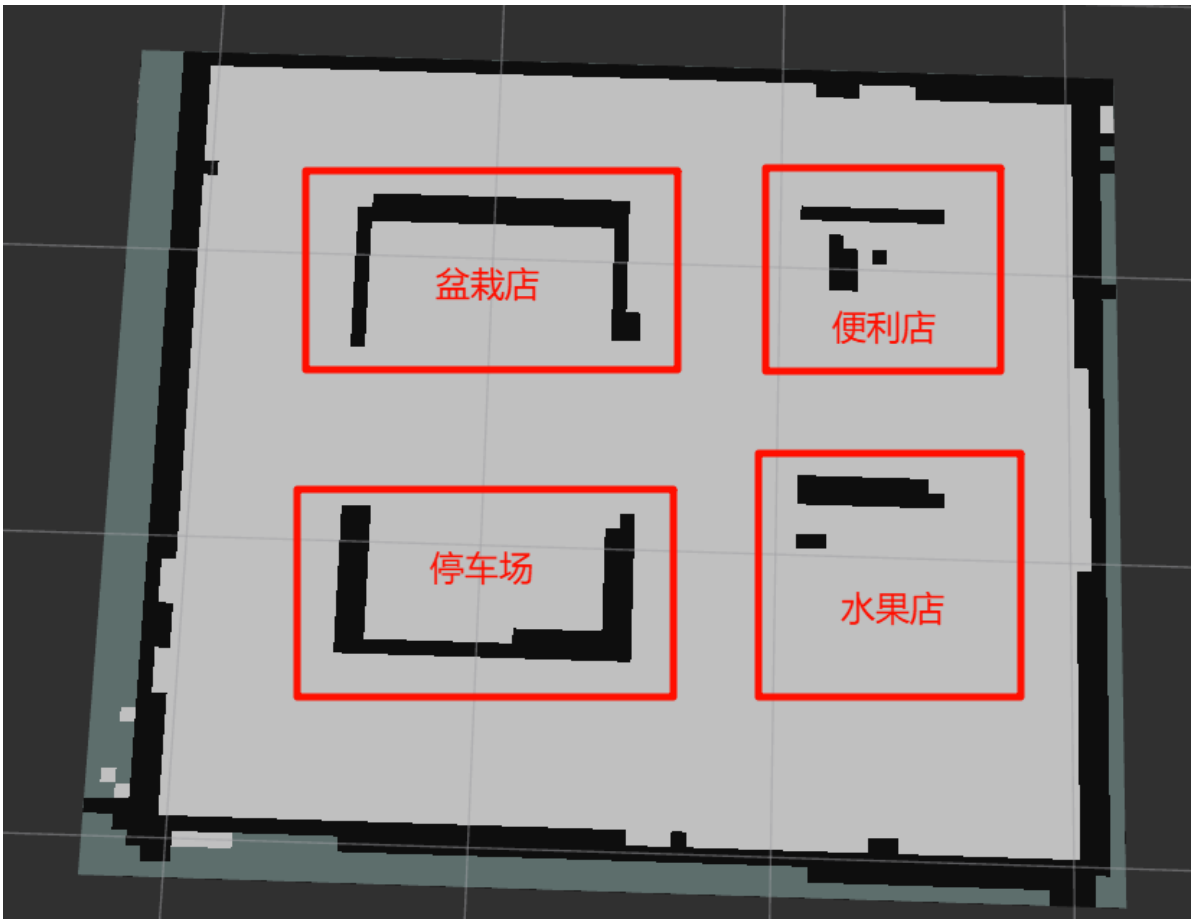


You can see the **/initialpose** topic. This topic was published using the **2D Pose Estimate** tool in rviz2 in the previous step. We can use this tool to publish coordinates and view the data of the **/initialpose** topic to obtain the coordinates and orientation angle of a point on the map.

Enter the following command in the terminal to observe the data on the **/initialpose** topic.

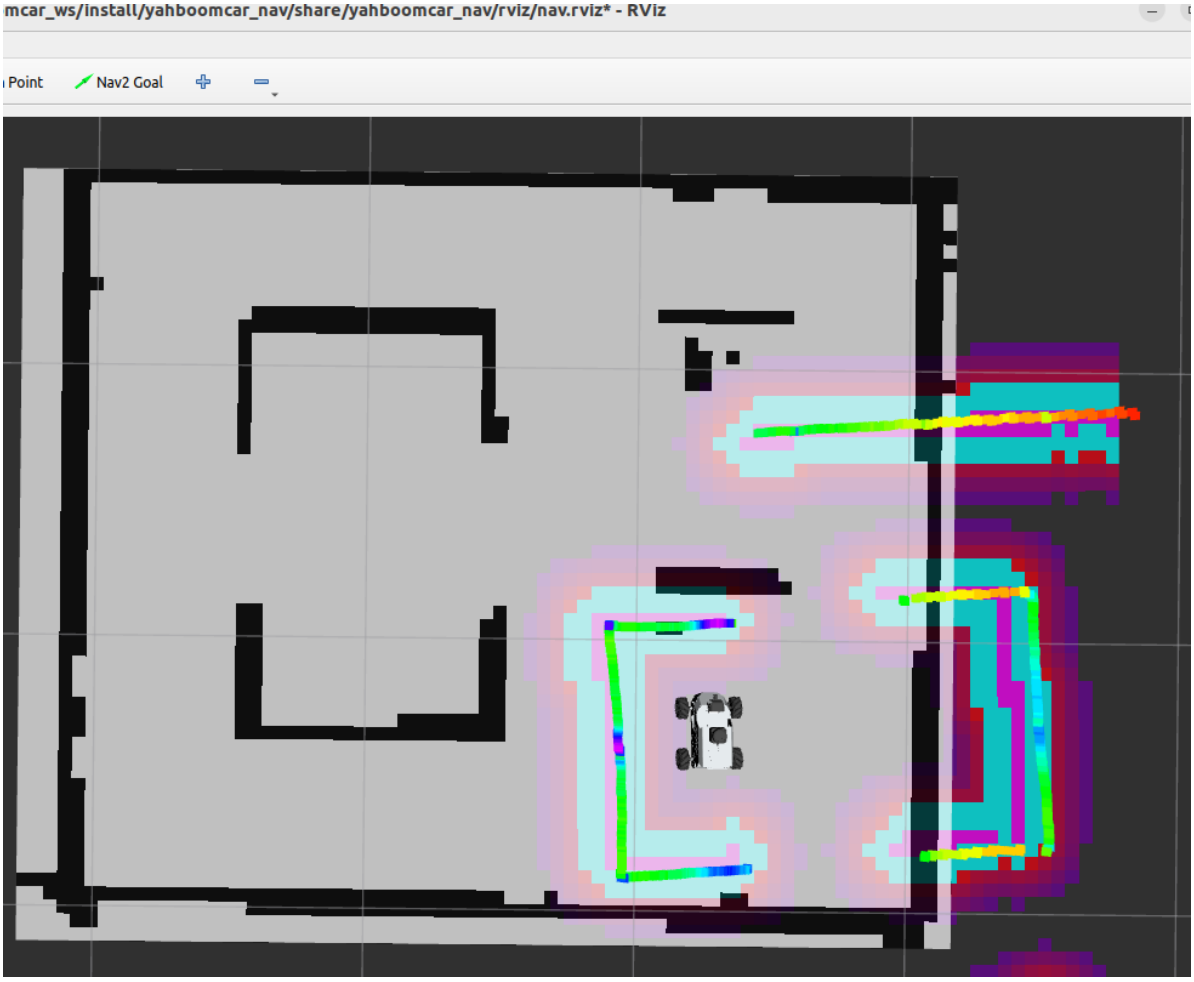
```
ros2 topic echo /initialpose
```

We can name any point on the map; here's an example of naming as shown in the image.

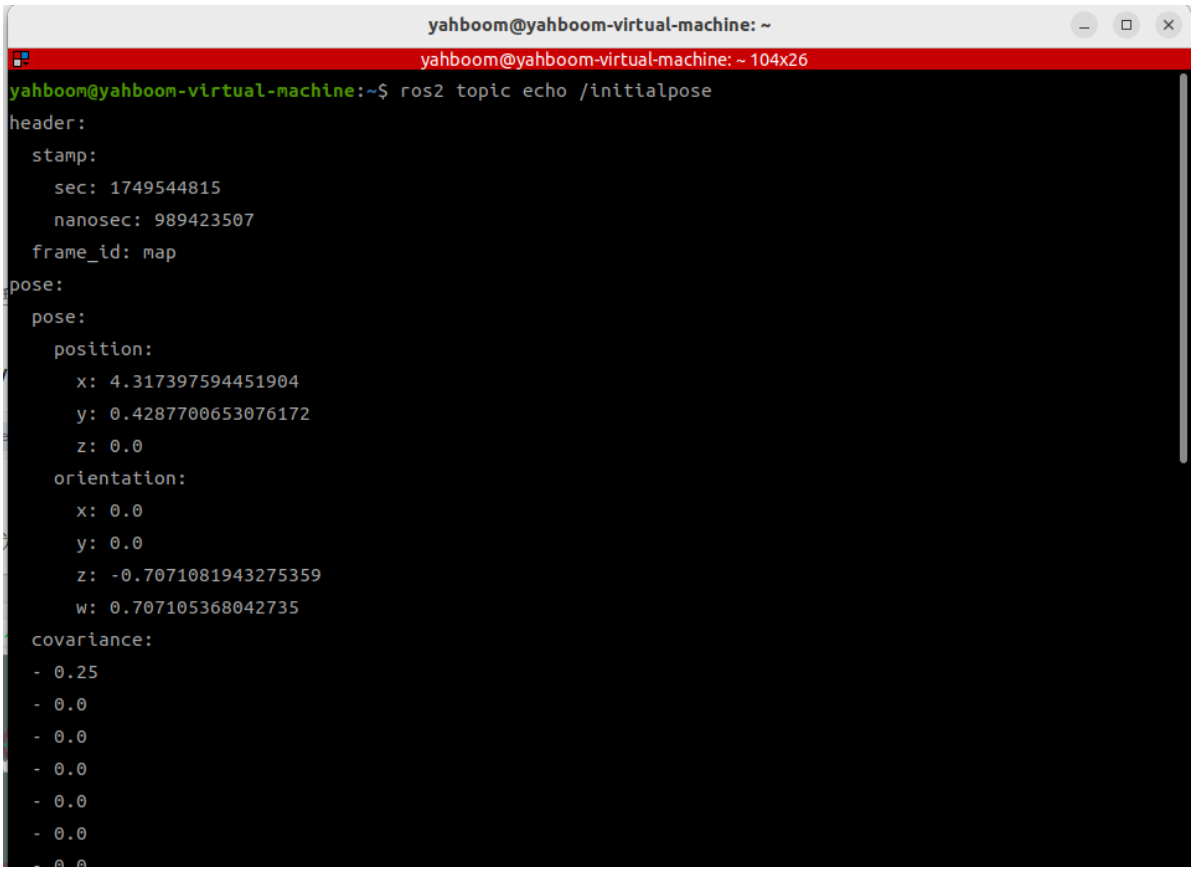


As shown in the image below, first click the **2D Pose Estimate** tool, then left-click and hold any location in the "Fruit Shop" area to adjust the orientation. Release the mouse button when you're satisfied.

The robot's position will be adjusted to the previously selected location and orientation. You can preview the robot's expected position and orientation in RViz.



The last frame of information in the terminal window shows the coordinates of the target point we just marked using the **2D Pose Estimate** tool.



Open the `map_mapping.yaml` map mapping file, located at:

jetson orin：

```
/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/config/map_mapping.yaml
```

Jetson Nano and Raspberry Pi (requires Docker access first)

```
/root/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains/config/map_mapping.yaml
```

Replace "name" in A with "Fruit Shop", "position", and "orientation" in the terminal with the data we just saw in the **/initialpose** topic.

```
#根据实际的场景环境，自定义地图中的区域，可以添加任意个区域，注意和大模型的地图映射保持一致即可
#According to the actual scene environment, customize the areas in the map. You can add any number of areas, just make sure they are consistent with the map mapping of the large model

#地图映射Map mapping

common_map_areas: #常规导航 common navigation
    A:
      name: '水果店'
      position:
        x: 1.339120626449585
        y: -0.4776395797729492
        z: 0.0
      orientation:
        x: 0.0
```

```
        y: 0.0
        z: 0.7038725577603971
        w: 0.7103262788548911

#此处可新增添加区域对应的栅格地图坐标点，注意和上面格式保持一致
#Here, you can add the raster map coordinate points corresponding to the added
area. Please note that the format should be consistent with the above
```

In the same way, we can add a map mapping for the "Potted Plant Shop".

```
#根据实际的场景环境，自定义地图中的区域，可以添加任意个区域，注意和大模型的地图映射保持一致即可
#According to the actual scene environment, customize the areas in the map. You
can add any number of areas, just make sure they are consistent with the map
mapping of the large model
#地图映射Map mapping

common_map_areas: #常规导航 common navigation
    A:
      name: '水果店'
      position:
        x: 1.339120626449585
        y: -0.4776395797729492
        z: 0.0
      orientation:
        x: 0.0
        y: 0.0
        z: 0.7038725577603971
        w: 0.7103262788548911
    B:
      name: '盆栽店'
      position:
        x: -0.0008649379014968872
        y: 0.4937915802001953
        z: 0.0
      orientation:
        x: 0.0
        y: 0.0
        z: 0.7133229968132297
        w: 0.7008354316224266
•
#此处可新增添加区域对应的栅格地图坐标点，注意和上面格式保持一致
#Here, you can add the raster map coordinate points corresponding to the added
area. Please note that the format should be consistent with the above
```

Then, switch to the `/yahboomcar_ros2_ws/yahboomcar_ws#/` workspace in the terminal: recompile the multi_brains package to apply the configuration. **(For Jetson Nano and Raspberry Pi hosts, you need to enter Docker first)**

```
cd ~/yahboomcar_ros2_ws/yahboomcar_ws/
```

Recompile the package:

```
colcon build --packages-select multi_brains
```

```
jetson@yahboom:~$ cd ~/yahboomcar_ros2_ws/yahboomcar_ws/
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$ colcon build --packages-select multi_brains
Starting >>> multi_brains
Finished <<< multi_brains [2.53s]

Summary: 1 package finished [3.40s]
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$
```

3. Run the Example

3.1 Starting the Program

For Raspberry Pi and Jetson-Nano boards, you need to enter the Docker container first. For Orin controllers, this is not necessary.

Open the terminal and enter the command:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

The following content will be displayed after initialization is complete.

```
[action_service_nuwa-15]    from pkg_resources import resource_stream, resource_exists
[action_service_nuwa-15] [INFO] [1766129486.356630338] [action_service_nuwa]: enable_route_nav: False
[action_service_nuwa-15] [INFO] [1766129486.726464447] [action_service_nuwa]: ROS_Action_Service Initialization completed
[model_service-14] [INFO] [1766129518.715920692] [model_service]: Dify LLM connection successful
[model_service-14] [WARN] [1766129518.719145287] [rcl.logging_rosout]: Publisher already registered for provided node name. I
f this is due to multiple nodes with the same name then all logs for that logger name will go out over the existing publisher
. As soon as any node with that name is destructed it will unregister the publisher, preventing any further logs for that nam
e from being published on the rosout topic.
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.rear
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.center_lfe
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.side
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.hdmi
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.hdmi
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.modem
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.modem
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.phoneline
[model_service-14] ALSA lib pcm.c:2664:(snd_pcm_open_noupdate) Unknown PCM cards.pcm.phoneline
[model_service-14] ALSA lib pcm_oss.c:397:(_snd_pcm_oss_open) Cannot open device /dev/dsp
[model_service-14] ALSA lib pcm_oss.c:397:(_snd_pcm_oss_open) Cannot open device /dev/dsp
[model_service-14] ALSA lib confmisc.c:160:(snd_config_get_card) Invalid field card
[model_service-14] ALSA lib pcm_usb_stream.c:482:(_snd_pcm_usb_stream_open) Invalid card 'card'
[model_service-14] ALSA lib confmisc.c:160:(snd_config_get_card) Invalid field card
[model_service-14] ALSA lib pcm_usb_stream.c:482:(_snd_pcm_usb_stream_open) Invalid card 'card'
[model_service-14] [INFO] [1766129519.225800065] [ASR_Detect]: The online ASR_Engine:paraformer-realtime-v2 initialization ha
s been completed
[model_service-14] [INFO] [1766129519.274575027] [model_service]: Tongyi TTS_Engine initialized successfully
[model_service-14] [INFO] [1766129519.277227745] [model_service]: ROS_LLM_Service Initialization completed
```

Start the virtual machine by creating a new terminal. **(For Raspberry Pi, and Jetson Nano, it is recommended to run the visualization on the virtual machine.)**

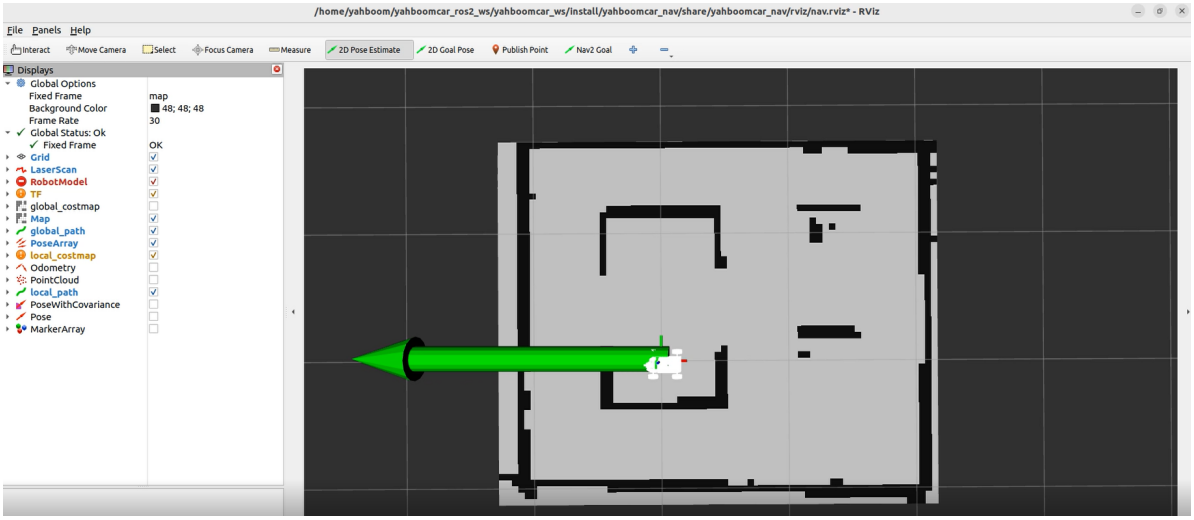
```
ros2 launch yahboomcar_nav display_nav_launch.py
```

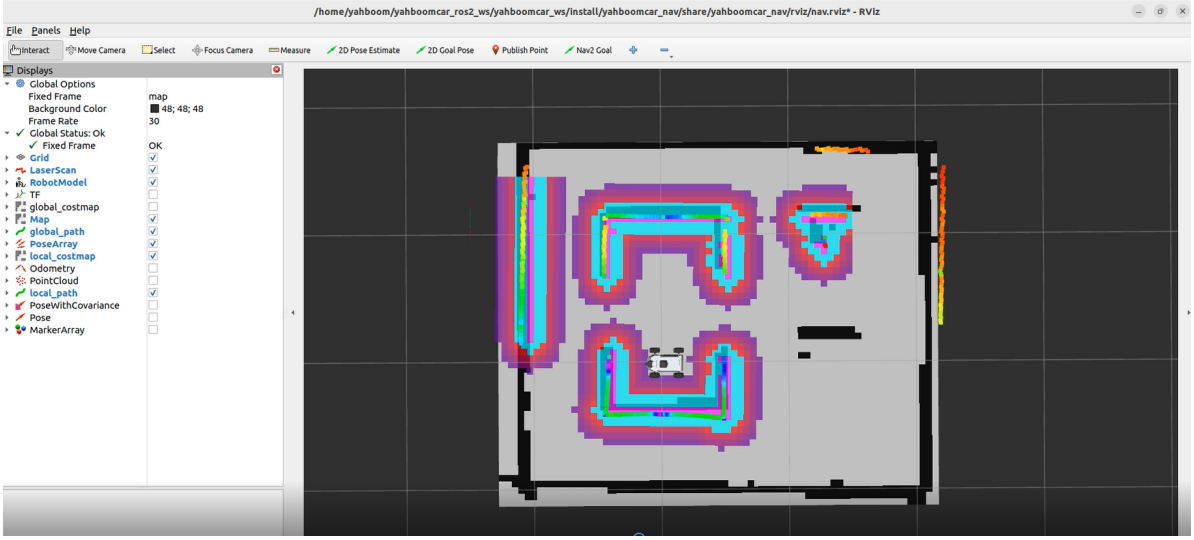
Wait for the navigation algorithm to activate and the map to appear, then open the terminal and enter...

```
# Choose one of the two navigation algorithms
# Standard navigation
ros2 launch yahboomcar_nav navigation_teb_launch.py

#Fast relocalization navigation ( Jetson nano and Raspberry Pi 5 are not supported)
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false load_state_filename:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.pbstream

ros2 launch yahboomcar_nav navigation_cartodwb_launch.py maps:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/yahboomcar.yaml params_file:=/home/jetson/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/params/cartoteb_nav_params.yaml
```

Note: yahboomcar.yaml and yahboomcar.pbstream must be mapped simultaneously, meaning they are the same map. Refer to the cartograph mapping algorithm for saving maps.

After that, follow the navigation function startup process to initialize positioning. The rviz2 visualization interface will open. Click **2D Pose Estimate** in the upper toolbar to enter the selection state. Roughly mark the robot's position and orientation on the map. After initializing positioning, preparations are complete.





3.2 Test Cases

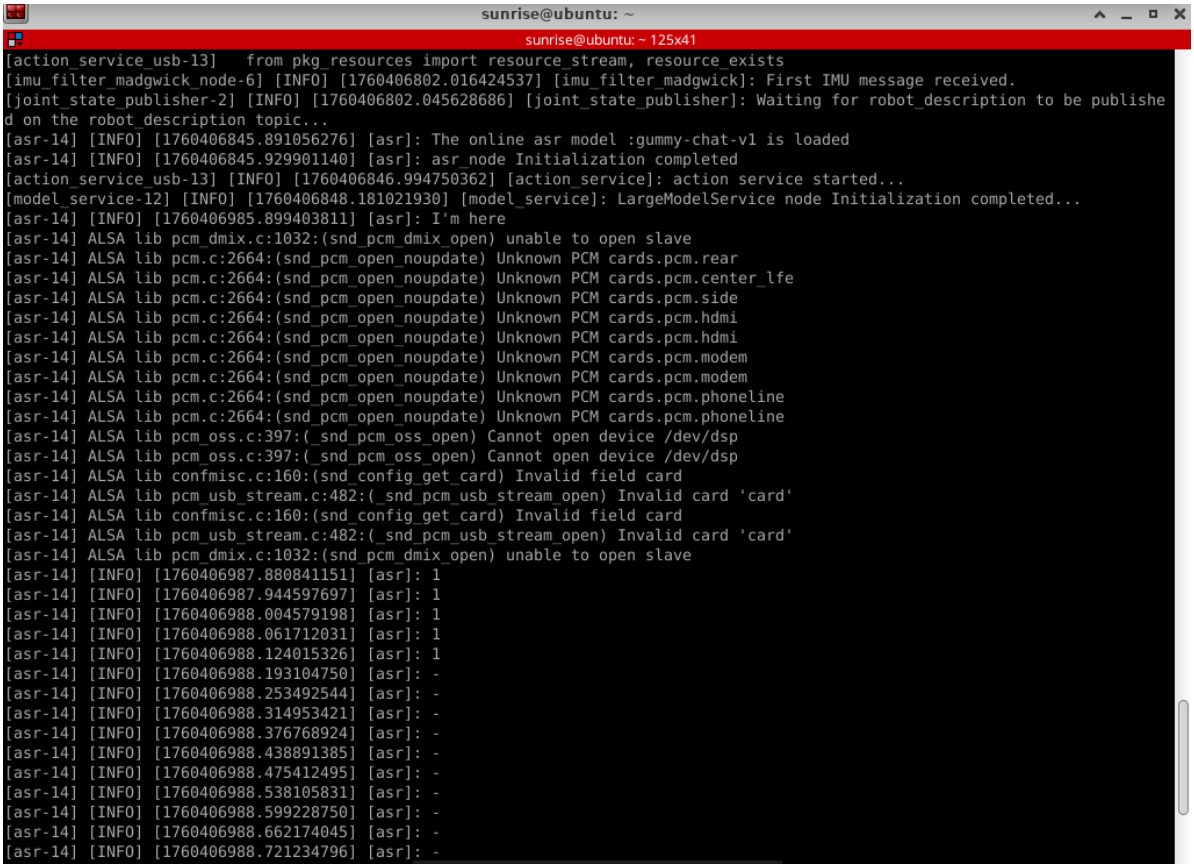
Here is a sample test case; users can create their own dialogue commands:

- Please remember your current location, then go to the fruit shop and the convenience store in sequence, return to your original location, and tell me what you saw.

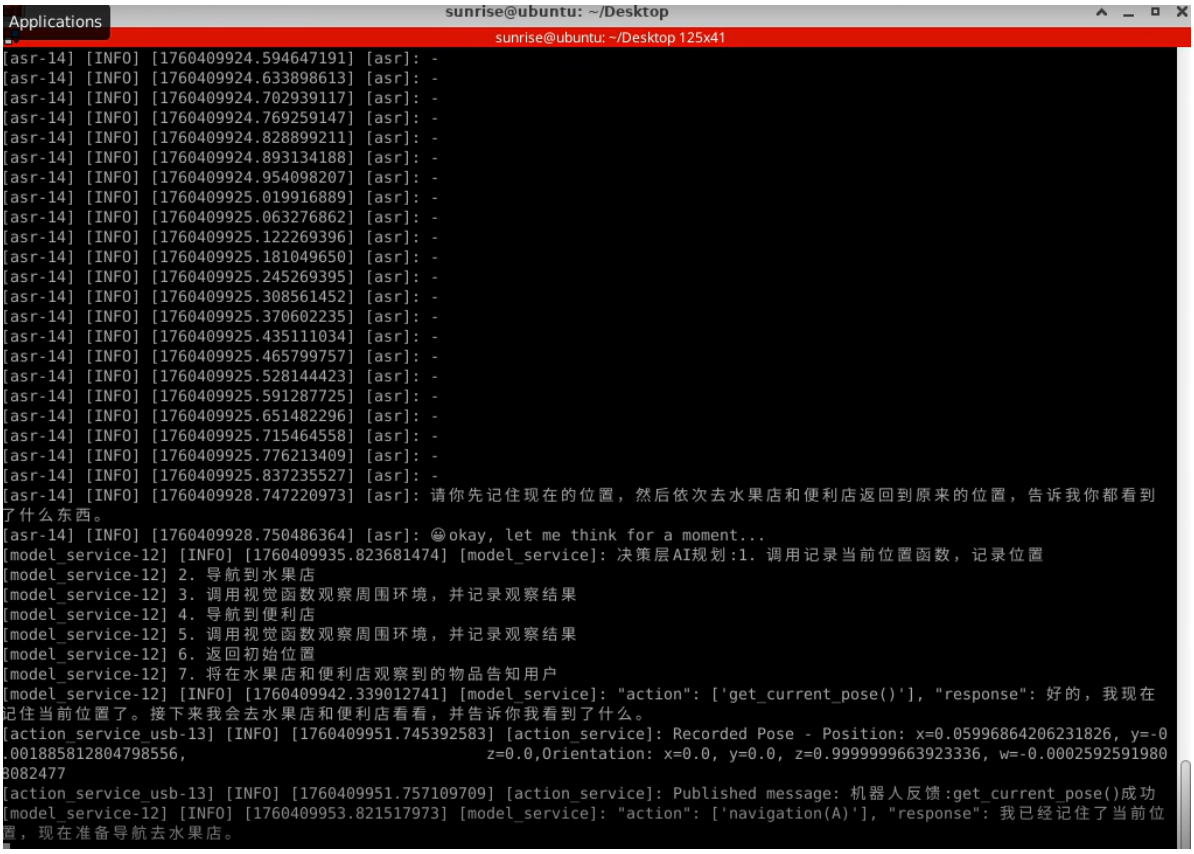
3.2.1 Case 1

First, wake up the robot by saying "Hello yahboom". The robot will respond, "I'm here, please give me order." After the robot responds, its buzzer will sound briefly (beep-), and then the user can speak. The robot will perform dynamic sound detection; if there is sound activity, it will print "1", and if there is no sound activity, it will print "-". After speaking, it will detect the end of the sound; if there is silence for more than 450ms, recording will stop.

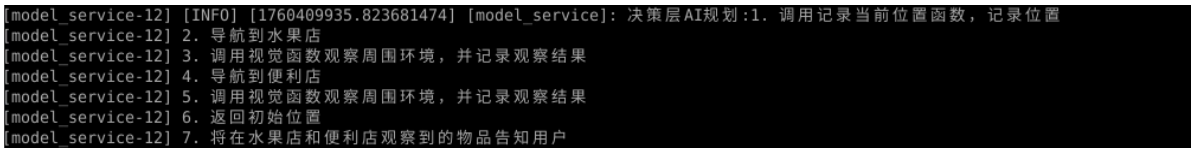
Dynamic Sound Detection (VAD) is shown in the following figure:



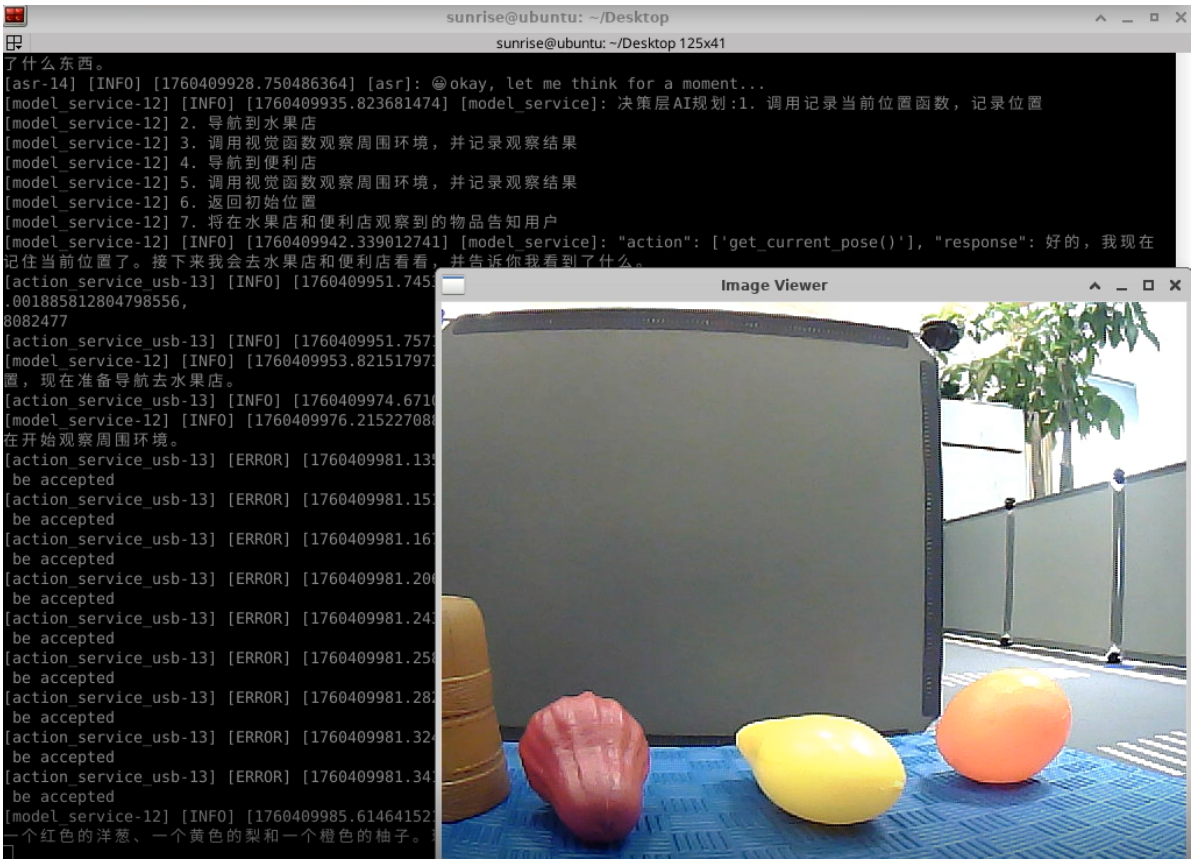
The robot will first interact with the user to get a response, then follow the instructions, and the terminal will print the following information:



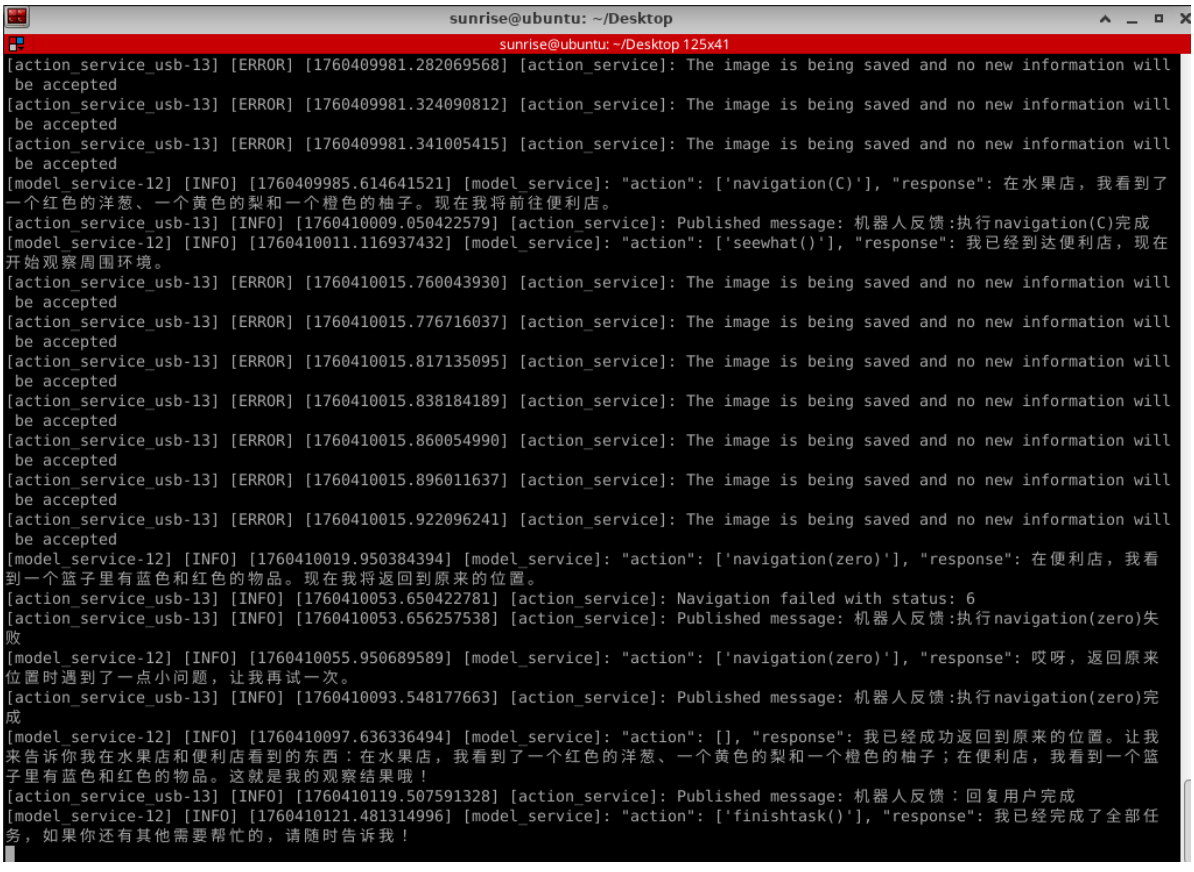
The large-scale model at the decision-making level outputs the planned task steps:



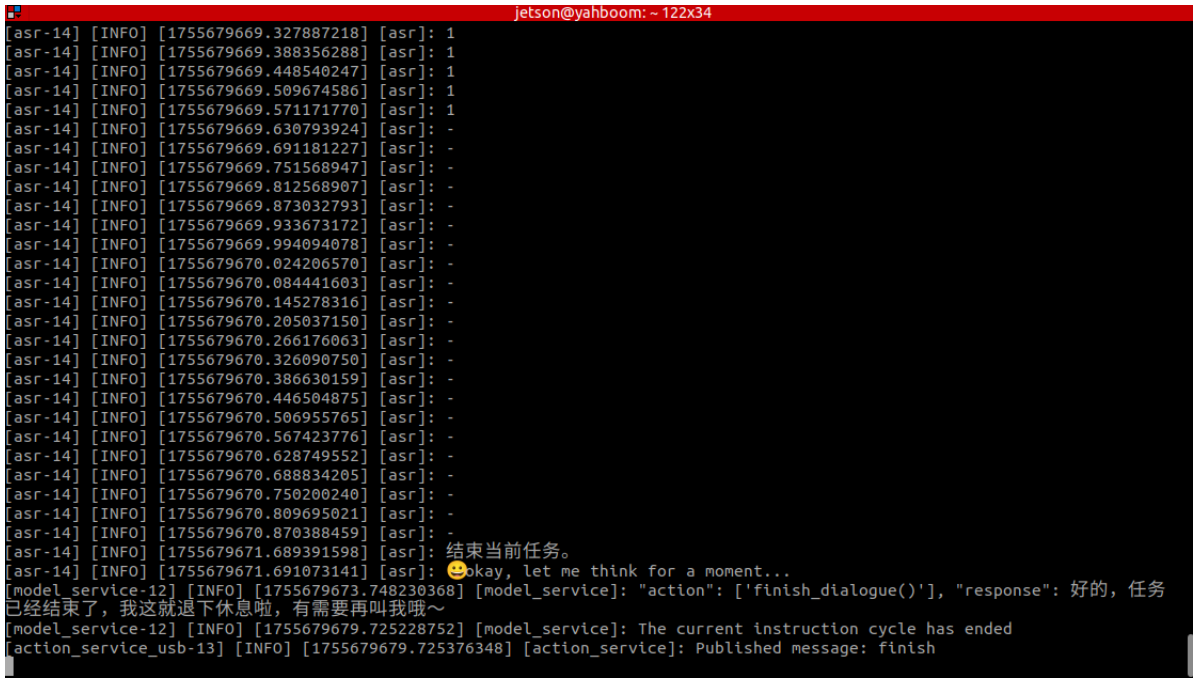
The execution layer will then follow the steps of this task:



Once the task is completed, wake up the robot by saying "Hello yahboom," and the robot will respond, "I'm here, please give me order." At this point, you can proceed with the next command.



After completing its task, the robot will enter a waiting state. At this point, the robot will re-enter a free-conversation state, but all conversation history will be retained. You can then wake yahboom again and "End Current Task" to terminate the current task cycle, clear the conversation history, and begin a new task cycle.


jetson@yahboom: ~ 122x34
[asr-14] [INFO] [1755679669.327887218] [asr]: 1
[asr-14] [INFO] [1755679669.388356288] [asr]: 1
[asr-14] [INFO] [1755679669.448540247] [asr]: 1
[asr-14] [INFO] [1755679669.509674586] [asr]: 1
[asr-14] [INFO] [1755679669.571171770] [asr]: 1
[asr-14] [INFO] [1755679669.630793924] [asr]: -
[asr-14] [INFO] [1755679669.691181227] [asr]: -
[asr-14] [INFO] [1755679669.751568947] [asr]: -
[asr-14] [INFO] [1755679669.812568907] [asr]: -
[asr-14] [INFO] [1755679669.873032793] [asr]: -
[asr-14] [INFO] [1755679669.933673172] [asr]: -
[asr-14] [INFO] [1755679669.994094078] [asr]: -
[asr-14] [INFO] [1755679670.024206570] [asr]: -
[asr-14] [INFO] [1755679670.084441603] [asr]: -
[asr-14] [INFO] [1755679670.145278316] [asr]: -
[asr-14] [INFO] [1755679670.205037150] [asr]: -
[asr-14] [INFO] [1755679670.266176063] [asr]: -
[asr-14] [INFO] [1755679670.326090750] [asr]: -
[asr-14] [INFO] [1755679670.386630159] [asr]: -
[asr-14] [INFO] [1755679670.446504875] [asr]: -
[asr-14] [INFO] [1755679670.506955765] [asr]: -
[asr-14] [INFO] [1755679670.567423776] [asr]: -
[asr-14] [INFO] [1755679670.628749552] [asr]: -
[asr-14] [INFO] [1755679670.688834205] [asr]: -
[asr-14] [INFO] [1755679670.750200240] [asr]: -
[asr-14] [INFO] [1755679670.809695021] [asr]: -
[asr-14] [INFO] [1755679670.870388459] [asr]: -
[asr-14] [INFO] [1755679671.689391598] [asr]: 结束当前任务。
[asr-14] [INFO] [1755679671.691073141] [asr]: 😀Okay, let me think for a moment...
[model_service-12] [INFO] [1755679673.748230368] [model_service]: "action": ['finish_dialogue()'], "response": 好的，任务已经结束了，我这就退下休息啦，有需要再叫我哦～
[model_service-12] [INFO] [1755679679.725228752] [model_service]: The current instruction cycle has ended
[action_service_usb-13] [INFO] [1755679679.725376348] [action_service]: Published message: finish